



SUPMEAL

Documentation technique

Plateforme de gestion de recettes et de planification de repas

Ce que couvre ce document

01

Architecture générale

Découpage 3-tiers, apps Django

02

Choix techniques

Justification backend / frontend / BDD

03

Base de données

Schéma relationnel et optimisations

04

Sécurité

Authentification, hashing, permissions

05

Diagrammes UML

Séquences d'authentification et de création

06

Déploiement

Guide Docker Compose et variables d'env.

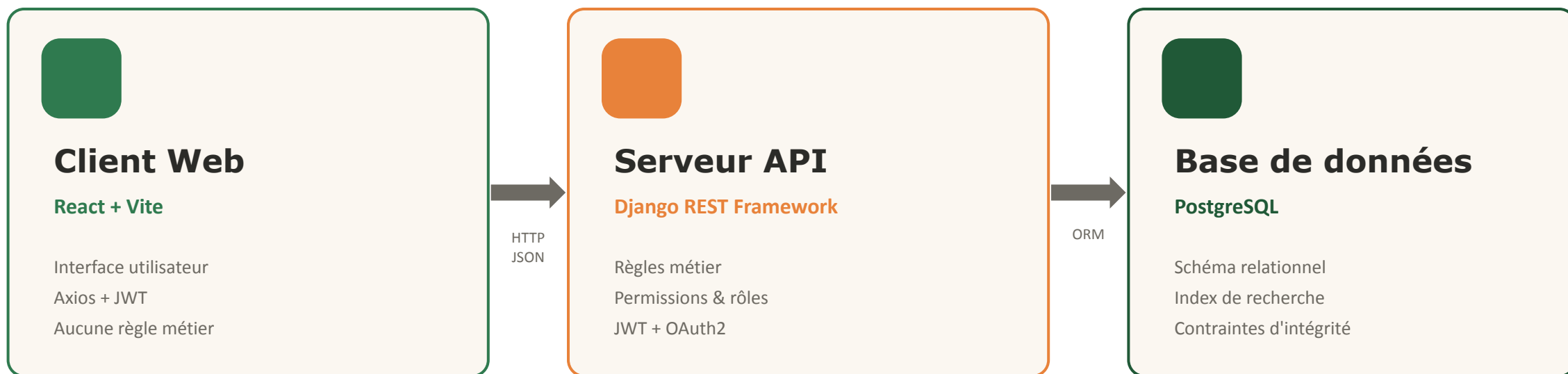
07

API & structure

Endpoints principaux et arborescence

Architecture générale — 3 briques distinctes

Aucune logique métier côté client : React n'est qu'une interface qui interroge l'API REST.



Conteneurisation : 3 services Docker distincts (backend, frontend, db) orchestrés via docker-compose.yml.

Modularité — 6 applications Django

1

accounts

Utilisateur custom, JWT, inscription, OAuth2, préférences culinaires

2

cookbooks

Cookbooks partagés, memberships, rôles, invitations

3

recipes

Recettes, ingrédients indexés, tags, favoris, commentaires, filtrage

4

planning

Planification de repas, génération de liste de courses

5

messaging

Messagerie instantanée interne à un cookbook

6

import/ export

Export/Import JSON (compatible Mealie) et CSV

Pourquoi Django REST Framework ?

Django

ORM robuste, migrations fiables, admin auto-généré pour le support/modération.

DRF

Viewsets, serializers et permissions déclaratifs : moins de duplication qu'en vues manuelles.

SimpleJWT

Authentification stateless (access + refresh token), adaptée à une API consommée par un SPA.

django-filter

Filtrage avancé déclaratif (2.2.4) sans dupliquer la logique de requête dans chaque vue.

Frontend & base de données

React + Vite

Vite : démarrage et build très rapides, configuration minimale.

React Router : navigation SPA classique.

Axios + intercepteurs : centralisent l'ajout du JWT et le rafraîchissement automatique du token.

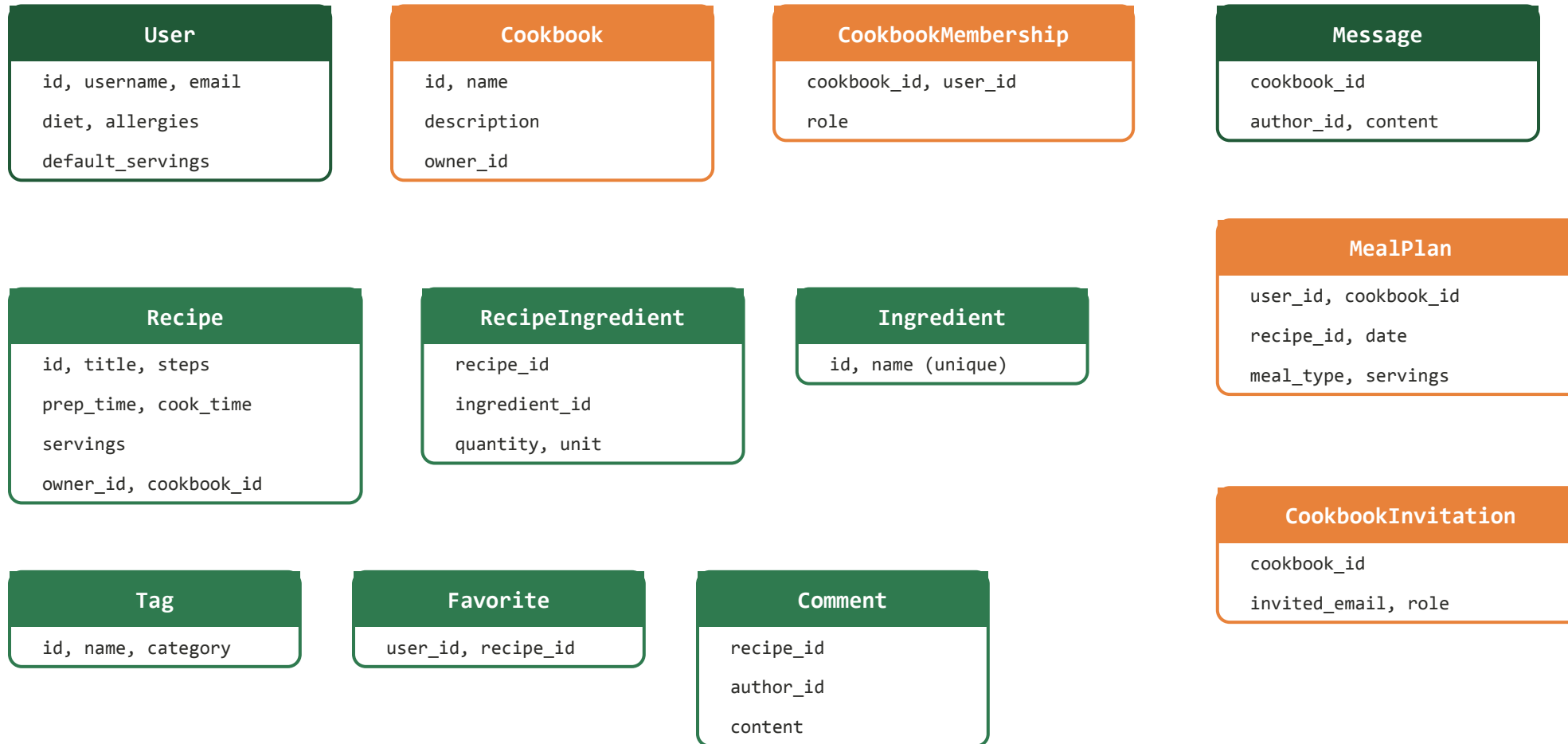
CSS custom : maîtrise totale de la charte graphique SUPMEAL.

PostgreSQL

SGBD relationnel robuste, adapté aux contraintes du modèle :

- Contrainte d'unicité (un rôle par utilisateur et par cookbook)
- Relations many-to-many (recettes ↔ tags, recettes ↔ favoris)
- Index sur les champs de recherche/filtrage fréquents
- Bascule automatique sur SQLite en développement local rapide

Schéma relationnel (simplifié)



Index & optimisation : table Ingredient centralisée (contrainte unique + index) é

Sécurité de l'application

1

Mots de passe hashés

Jamais stockés en clair : hashing PBKDF2 via `set_password()` (Django).

2

JWT signés

Access token à durée de vie courte, rotation du refresh token.

3

Permissions serveur

Chaque action sensible (édition, suppression, invitation, rôle) est vérifiée côté API — jamais côté client seul.

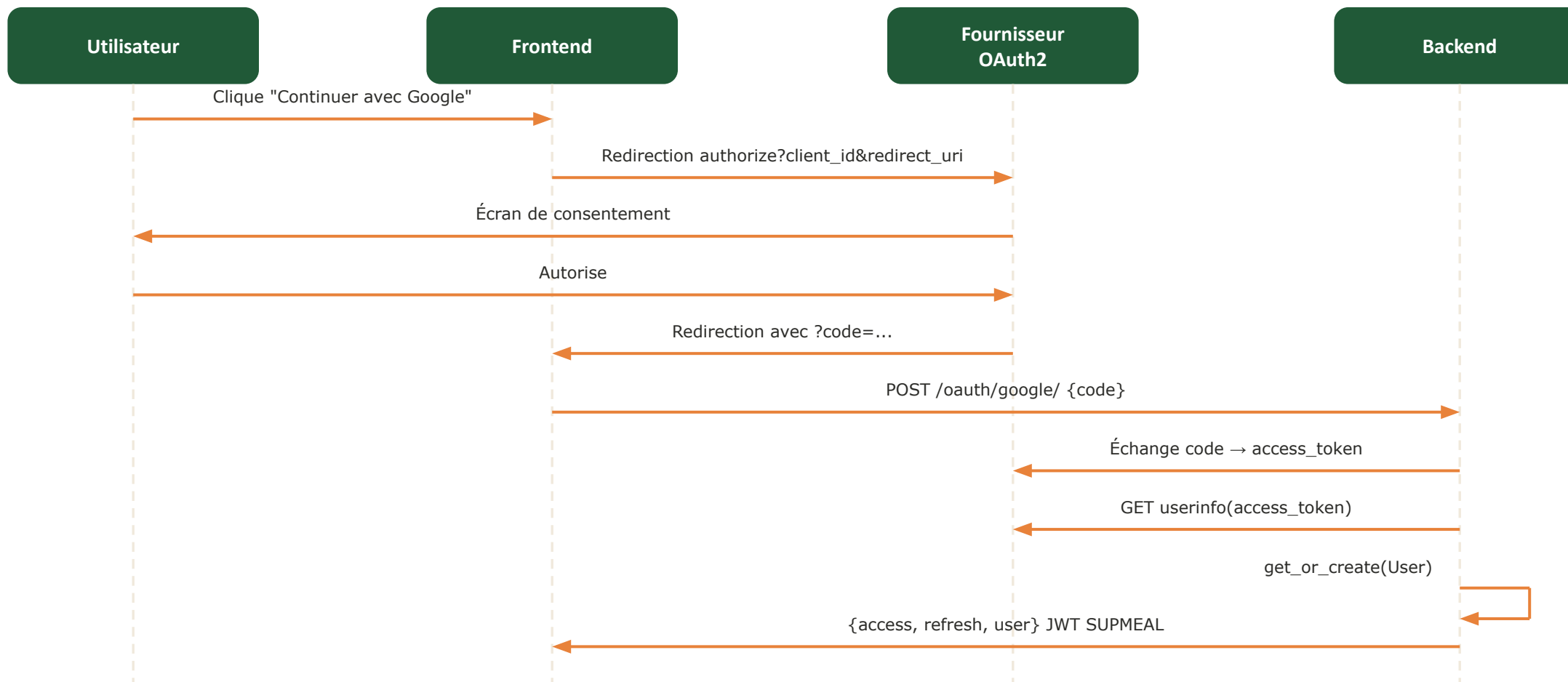
4

Secrets externalisés

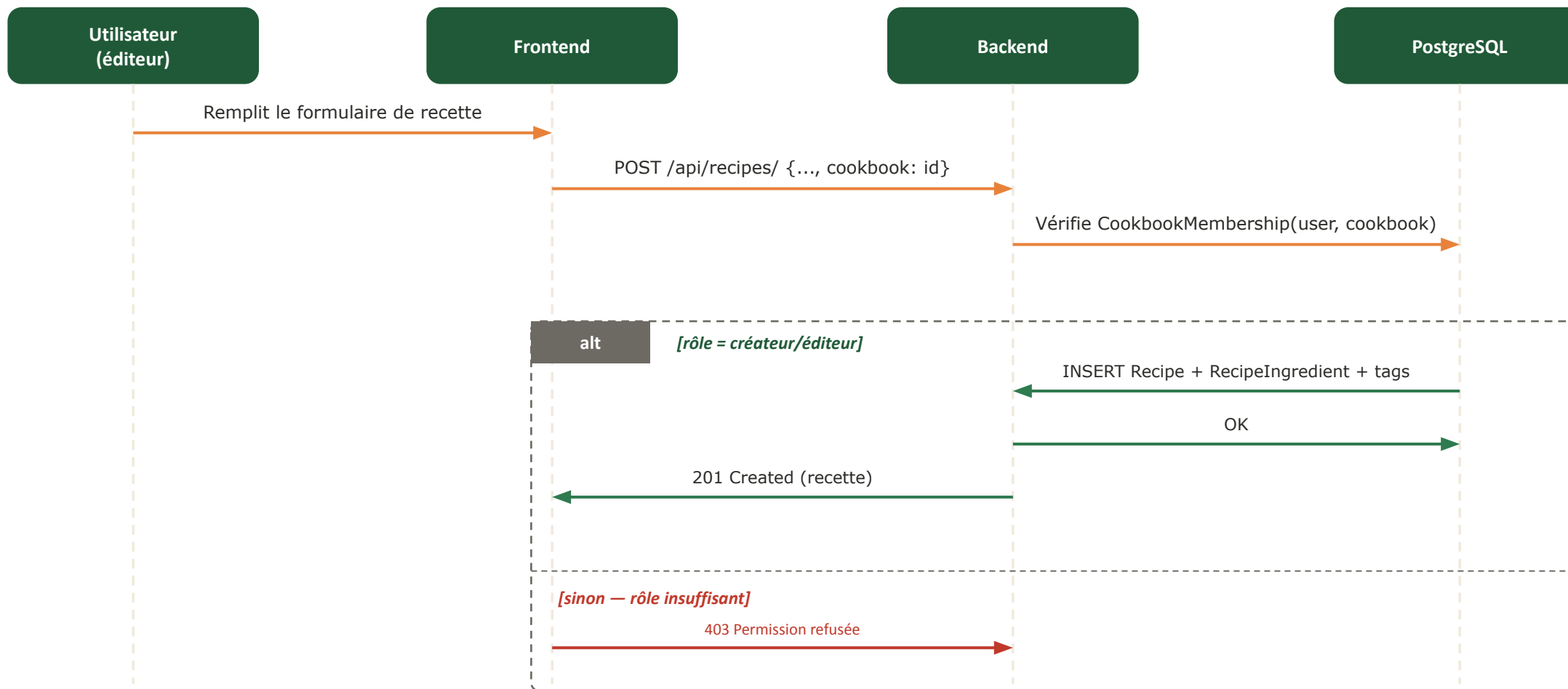
`SECRET_KEY`, identifiants OAuth2, mots de passe DB fournis via variables d'environnement (`.env` non versionné).

CORS maîtrisé : `CORS_ALLOWED_ORIGINS` restreint en production — le mode ouvert (`CORS_ALLOW_ALL`) est réservé au développement.

Séquence — Authentification OAuth2



Séquence — Ajout d'une recette dans un cookbook partagé



Guide de déploiement

1

Cloner le dépôt

```
git clone https://github.com/seraphingambo697/SUPMAIL.git  
&& cd supmeal
```

2

Configurer l'environnement

```
cp .env.example .env (éditer si besoin, ex. clés OAuth2)
```

3

Lancer l'application

```
docker compose up --build
```

Accès une fois lancé :

Frontend `http://localhost:3000`

API backend `http://localhost:8000/api/`

Admin Django `http://localhost:8000/admin/` (admin / admin1234 par défaut)

Variables d'environnement principales

Variable	Description
DJANGO_SECRET_KEY	Clé secrète Django (obligatoire en production)
DJANGO_DEBUG	True / False
POSTGRES_DB / USER / PASSWORD	Identifiants base de données
CORS_ALLOWED_ORIGINS	Origines autorisées à appeler l'API
GOOGLE_CLIENT_ID / SECRET	App OAuth2 Google — redirect URI : /oauth/callback/google
GITHUB_CLIENT_ID / SECRET	OAuth App GitHub
MICROSOFT_CLIENT_ID / SECRET	App Azure AD

Endpoints principaux de l'API

Méthode	Endpoint	Description
POST	/api/auth/register/	Inscription
POST	/api/auth/login/	Connexion (JWT)
POST	/api/auth/oauth/<provider>/	Connexion OAuth2
GET/POST	/api/cookbooks/	Lister / créer un cookbook
POST	/api/cookbooks/<id>/invite/	Inviter un membre
GET/POST	/api/recipes/?tag=&ingredient=&favorite=	Lister / filtrer / créer des recettes
POST	/api/recipes/<id>/toggle_favorite/	Ajouter/retirer des favoris
GET	/api/meal-plans/shopping_list/	Liste de courses générée
GET/POST	/api/messages/?cookbook=	Messagerie du cookbook
GET	/api/export/?export_format=json csv	Export des données
POST	/api/import/	Import de recettes/cookbooks

Structure du dépôt

```
supmeal/  
├── docker-compose.yml  
├── .env.example  
├── backend/  
│   ├── manage.py · requirements.txt · Dockerfile  
│   ├── supmeal/          (settings, urls, wsgi/asgi)  
│   └── accounts / cookbooks / recipes / planning /  
│       messaging / importexport  
├── frontend/  
│   ├── src/api/          (client axios + refresh JWT)  
│   ├── src/context/      (AuthContext)  
│   ├── src/components/   (Layout, RecipeCard...)  
│   ├── src/pages/        (Login, Dashboard, Recipes...)  
│   └── package.json · Dockerfile  
├── README.md             (documentation technique)  
└── USER_MANUAL.md        (manuel utilisateur)
```

Merci

Documentation technique SUPMEAL

Voir aussi le manuel utilisateur.